



**REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: ingeniería informática
UNIDAD CURRICULAR: Lenguajes y Compiladores**

Tema 3: Lenguajes y Gramáticas Formales

NOMBRE DEL DOCENTE: Félix Márquez

NOMBRE DE ESTUDIANTES:

José Silva, V-30.810.283 SEC: 2

Nicole Serano, V-31.275.411 SEC: 2

Eloismary Díaz, V-28.682.879 SEC: 2

José Moreno, V-28.086.251 SEC: 2

CIUDAD GUAYANA, 15– 06 – 2026

Introducción

La teoría de la computación y los lenguajes formales constituyen el núcleo matemático y lógico sobre el cual se erigen las ciencias de la computación modernas. En su nivel más fundamental, un lenguaje formal proporciona un marco riguroso para definir la sintaxis y la estructura de la información, permitiendo que las máquinas procesen, compilen e interpreten datos de manera determinista. Para lograr esto, la disciplina se apoya en modelos generativos, conocidos como gramáticas, y en modelos de reconocimiento, definidos como autómatas.

El presente informe tiene como objetivo fundamental explorar la intersección entre la teoría abstracta de lenguajes y su aplicación directa en el desarrollo de software y la ingeniería de compiladores. Para abarcar esta transición de la teoría a la práctica

A través de la combinación de bases teóricas y simulaciones algorítmicas en JavaScript, este informe demuestra cómo las abstracciones matemáticas se traducen en motores de validación, procesadores gráficos y analizadores léxicos funcionales.

1.- Fundamentos y Jerarquía.

1.1.- Relación Gramática-Lenguaje.

Concepto de la relación entre Gramáticas y Lenguajes Formales

En la teoría de la computación, un lenguaje formal es un conjunto de cadenas de símbolos (palabras) seleccionadas de un alfabeto determinado (Sipser, 2013). Debido a que muchos lenguajes contienen un número infinito de cadenas, es imposible enumerarlas todas de forma explícita. Es en este punto donde surge la necesidad de una gramática formal, la cual se define como un modelo matemático o conjunto de reglas estructurales abstractas que describe y delimita con total precisión matemática qué cadenas pertenecen a dicho lenguaje y cuáles no (Hopcroft y Ullman, 2006).

La relación entre ambas es de carácter generativo: la gramática actúa como el "molde" o el "algoritmo axiomático" ejecutable, mientras que el lenguaje es el producto o el conjunto resultante de todas las estructuras válidas que se pueden derivar de ella. De este modo, un lenguaje formal se denota matemáticamente como $L(G)$, que significa "el lenguaje generado por la gramática G ".

Mecanismo de generación de un lenguaje mediante una gramática

Una gramática formal se define formalmente como una estructura de 4-tuplas:

- $G = (V, \Sigma, P, S)$ (Chomsky, 1959; Linz, 2016):
- V (Variables o No terminales): Símbolos abstractos que no aparecen en las cadenas finales, pero ayudan a estructurarlas.
- Σ (Terminales): El alfabeto real del lenguaje; los caracteres finales de las cadenas.
- P (Reglas de producción): Relaciones de reemplazo de la forma $\alpha \rightarrow \beta$.
- S (Símbolo inicial): El elemento de partida dentro de las variables ($S \in V$).

El mecanismo por el cual una gramática genera una cadena se denomina derivación. El proceso inicia de manera estricta en el símbolo inicial S . A partir de allí, se selecciona secuencialmente una regla de producción adecuada de P para sustituir un símbolo no terminal por la cadena equivalente definida en la regla (Martin, 2011). Este reemplazo sucesivo se repite paso a paso y se representa mediante el operador de derivación $\rightarrow$.

El proceso de derivación concluye de manera exitosa únicamente cuando la cadena de símbolos está compuesta por completo por elementos pertenecientes al alfabeto de terminales Σ , eliminando por completo cualquier variable intermedio (Harrison, 1978). El conjunto de todas las cadenas posibles compuestas solo por terminales que pueden derivarse desde S constituye formalmente el lenguaje de dicha gramática.

Ejemplo Práctico de Relación

Supongamos una gramática diseñada para generar expresiones binarias simétricas básicas:

Variables: $V = \{S\}$

Terminales: $\Sigma = \{0, 1\}$

Reglas de producción P :

$S \rightarrow 0 S 0$

$S \rightarrow 1 S 1$

$S \rightarrow \epsilon$ (donde ϵ representa la cadena vacía)

Si aplicamos el mecanismo de derivación consecutiva partiendo desde S :

$S \rightarrow 0S0 \rightarrow 01S10 \rightarrow 01\epsilon10 = 0110$

La gramática ha generado con éxito la palabra 0110. El lenguaje completo $L(G)$ de esta gramática representará matemáticamente el conjunto infinito de todos los palíndromos binarios de longitud par (Hopcroft y Ullman, 2006).

1.2.- Jerarquía de Chomsky.

La Jerarquía de Chomsky clasifica las gramáticas formales en cuatro niveles según las restricciones impuestas a sus reglas de producción de la forma $\alpha \rightarrow \beta$, donde cada nivel representa una clase de lenguajes con capacidades de cómputo y reconocimiento bien diferenciadas (Chomsky, 1956).

Tipo 0: Gramáticas Sin Restricciones:

No imponen límites en el diseño de las producciones. La parte izquierda (α) debe contener al menos un símbolo no terminal, pero tanto α como β pueden ser cualquier combinación de terminales y variables (Linz, 2016). Estas gramáticas poseen la máxima capacidad expresiva computacional y describen los lenguajes recursivamente enumerables, los cuales son reconocidos por una Máquina de Turing universal (Sipser, 2013).

Ejemplo Práctico en Notación BNF: Representación conceptual de un procesador de transformaciones arbitrarias

`<inicio> ::= <contexto_A> "x"`

`<contexto_A> "x" ::= "y" <contexto_B>`

`<contexto_B> ::= "z"`

Tipo 1: Gramáticas Sensibles al Contexto

Se caracterizan porque la sustitución de una variable depende de los símbolos que la rodean. Formalmente, toda regla tiene la estructura $\alpha A \beta \rightarrow \alpha \gamma \beta$, o bien bajo la condición matemática de longitud no decreciente $|\beta| \geq |\alpha|$ (Martin, 2011). Un símbolo no terminal cambia solo dentro de su "contexto". Generan los lenguajes sensibles al contexto, reconocidos por un Autómata Linealmente Acotado (Harrison, 1978). Su caso de uso clásico es validar estructuras donde se requiere contar tres elementos idénticos en paralelo (como el lenguaje a^n, b^n, c^n).

Ejemplo Práctico en Notación BNF: Sintaxis conceptual dependiente del entorno circundante

```
<variable_A> "declarada" ::= "entero" <variable_A> "declarada"
```

```
<variable_A> ::= "identificador"
```

Tipo 2: Gramáticas Libres de Contexto

Dictaminan que la parte izquierda de toda regla de producción debe contener de forma exclusiva un único símbolo no terminal ($\alpha \in V$), sin importar qué caracteres aparezcan a su alrededor (Chomsky, 1959). Es decir, las sustituciones se realizan de manera completamente independiente del contexto espacial. Definen los lenguajes libres de contexto, esenciales para modelar la sintaxis de los lenguajes de programación y las estructuras de datos anidadas, y son reconocidos mediante un Autómata con Pila (Aho y Ullman, 2007).

Ejemplo Práctico en Notación BNF: Estructura clásica de un bucle condicional if-else

```
<sentencia_if> ::= "if" "(" <condicion> ")" <bloque> | "if" "(" <condicion> ")" <bloque> "else"
<bloque>
```

```
<condicion> ::= "true" | "false"
```

$\langle \text{bloque} \rangle ::= \{ "sentencia" \}$

Tipo 3: Gramáticas Regulares

Son las más restrictivas de la jerarquía. Sus reglas deben ser estrictamente lineales a la derecha (de la forma $A \rightarrow \alpha B$ o $A \rightarrow \alpha$) o lineales a la izquierda (Aho y Ullman, 2007). Esto imposibilita el anidamiento indefinido o el conteo de elementos emparejados. Generan los lenguajes regulares, empleados masivamente para el análisis léxico de tokens en compiladores y expresiones regulares, y son reconocidos por un Autómata Finito (Sipser, 2013).

Ejemplo Práctico en Notación BNF: Definición regular exacta de un número identificador de sistema

$\langle \text{id_sistema} \rangle ::= \text{"id_"} \langle \text{digitos} \rangle$

$\langle \text{digitos} \rangle ::= \text{"0"} \langle \text{digitos} \rangle \mid \text{"1"} \langle \text{digitos} \rangle \mid \text{"0"} \mid \text{"1"}$

2.- Derivación y Modelado.

2.1.- Gramática para dibujar: Basado en una Gramática Libre de Contexto (GLC)

Crear dibujos y figuras geométricas en una computadora usando texto se basa en teorías matemáticas que estudian cómo cambian las formas de las células y cómo funciona el diseño digital (Rozenberg y Salomaa, 1997). Es un sistema donde muchas reglas de escritura se aplican al mismo tiempo para transformar el texto en imágenes.

Para lograr que la estructura del ADN se conecte directamente con las herramientas de dibujo basadas en instrucciones automáticas, los autores crearon un sistema de reglas estrictas llamado Gramática Libre de Contexto (GLC), al cual nombraron $G_{dibujar} = (V, \Sigma, P, S)$.

En este sistema, el grupo de letras básicas que se pueden usar (llamado alfabeto Σ) imita a los cuatro componentes principales del ADN. Sin embargo, en lugar de servir para la biología, estas cuatro letras tienen un significado diferente para la computadora: funcionan como órdenes específicas (como "avanza" o "gira") para mover un puntero gráfico en la pantalla, (Hopcroft y Ullman, 2006).

Componentes de la Gramática

- Variables (No Terminales): $V = \{S, Cuadrado, \text{Árbol}, Cubo, Escalera, Rectángulo, A\}$
- Alfabeto de Terminales: $\Sigma = \{a, c, g, t\}$
- Símbolo Inicial: S

Mapeo Semántico del Alfabeto Genómico (Σ)

Para que las cadenas derivadas generen figuras coherentes, el motor de interpretación traduce los terminales bajo las siguientes reglas operacionales de dibujo lineal (Linz, 2016):

- Avanzar: Desplaza el cursor hacia adelante una unidad de longitud fija manteniendo el trazo activo pintando una línea.
- Girar Derecha: Modifica el ángulo de orientación del cursor rotando exactamente 90 grados en sentido horario.
- Girar Izquierda: Modifica el ángulo de orientación del cursor rotando exactamente 90 grados en sentido contra las agujas del reloj.

- Bifurcación: Operador de bifurcación geométrica o profundidad. Guarda el estado actual en una pila, ejecuta un desplazamiento alternativo (p. ej., arista tridimensional oblicua a 45° o ramificación foliar) y restaura el estado original al finalizar la instrucción.

Conjunto de Reglas de Producción (*P*)

1. $S \rightarrow \text{Cuadrado} \mid \text{Arbol} \mid \text{Cubo} \mid \text{Escalera} \mid \text{Rectangulo}$
2. $\text{Cuadrado} \rightarrow \text{acacacac}$
3. $\text{Rectangulo} \rightarrow \text{aacacaacac}$
4. $\text{Escalera} \rightarrow \text{acagA}$
5. $A \rightarrow \text{acagA} \mid a$
6. $\text{Arbol} \rightarrow \text{agArbolcArbolt} \mid \text{at}$
7. $\text{Cubo} \rightarrow \text{CuadradoCuadrado}$

2. Ejemplos de Derivación Estructural

A continuación se exponen las derivaciones matemáticas paso a paso aplicando el operador de producción ($\rightarrow$) desde el axioma *S* hasta la obtención de las sentencias genómicas terminales que modelan las figuras solicitadas (Sipser, 2013).

- **Caso A: Modelado de un Cuadrado**

Este caso geométrico requiere el trazo de cuatro aristas de idéntica longitud separadas por ángulos ortogonales rectos hacia la derecha.

$S \rightarrow \text{cuadrado}$

$\rightarrow \text{acacacac}$

Resultado terminal: acacacac

- **Caso B: Modelado de un Rectángulo**

Modifica la proporción dimensional de las aristas opuestas duplicando la longitud del avance inicial (aa).

$$\begin{aligned} S &\rightarrow \text{Rectangulo} \\ &\rightarrow \text{aacacaacac} \end{aligned}$$

Resultado terminal: aacacaacac

- **Caso C: Modelado de una Escalera (3 Peldaños)**

Aplica una estructura recursiva por la derecha mediante la variable A para intercalar avances horizontales y verticales utilizando giros opuestos (c y g).

$$\begin{aligned} S &\rightarrow \text{Escalera} \\ &\rightarrow \text{acagA} \\ &\rightarrow \text{acagacagA} \\ &\rightarrow \text{acagacaga} \end{aligned}$$

Resultado terminal: acagacaga

- **Caso D: Modelado de un Árbol Fractal con Ramas y Hojas**

Representa una ramificación recursiva libre de contexto. Cada iteración despliega un tronco, una desviación a la izquierda (g), un subárbol, una desviación simétrica a la derecha (c) y finaliza coronando la estructura con una hoja o nodo terminal (t).

$$\begin{aligned} S &\rightarrow \text{Arbol} \\ &\rightarrow \text{agArbolcArbolt} \\ &\rightarrow \text{ag(at)c(at)t} \\ &\rightarrow \text{agatcatt} \end{aligned}$$

Resultado terminal: agatcatt

- **Caso E: Modelado de un Cubo Tridimensional**

Modela la interconexión de dos planos bidimensionales cuadrangulares desfasados en el espacio mediante el operador de transformación y profundidad t, el cual proyecta las aristas de perspectiva oblicua.

$S \rightarrow \text{Cubo}$

$\rightarrow \text{Cuadrado} \cdot \text{Cuadrado}$

$\rightarrow \text{acacacactCuadrado}$

$\rightarrow \text{acacacatacacacac}$

Resultado terminal: acacacactacacacac

3.- Higiene y Optimización de Gramáticas.

3.1.- Patologías de las Gramáticas

Las gramáticas formales empleadas en la construcción de compiladores deben poseer una estructura que permita un análisis sintáctico determinista y eficiente. Cuando una gramática está mal diseñada, presenta patologías que impiden la generación de árboles de derivación únicos o provocan ciclos infinitos en los algoritmos de análisis descendente. Se ejemplificarán con 3 casos prácticos diferentes:

a) Gramática Ambigua

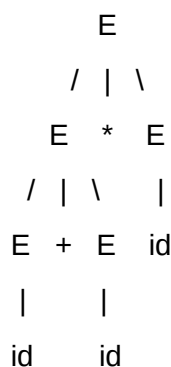
Una gramática se considera ambigua si es capaz de generar una misma cadena de terminales a través de dos o más árboles de derivación distintos, lo que equivale a decir que la cadena posee dos o más derivaciones por la izquierda diferentes. Esto representa un problema crítico en los lenguajes de programación, ya que el compilador no puede determinar de forma única la estructura o el significado semántico del programa.

Ejemplo Práctico:

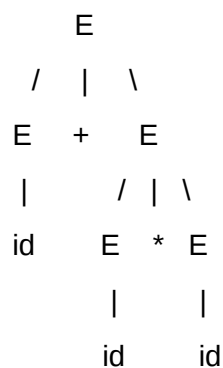
Consideremos la gramática simplificada para expresiones aritméticas

$E \rightarrow E + E \mid E * E \mid id$ y la cadena $id + id * id$.

Árbol de derivación 1 (Precedencia incorrecta, agrupa la suma primero):



Árbol de derivación 2 (Precedencia correcta, agrupa la multiplicación primero):



Dado que la misma gramática genera dos estructuras distintas para la misma cadena, queda demostrada su ambigüedad.

b) Recursividad por la Izquierda

Una gramática es recursiva por la izquierda si posee un símbolo no terminal A que puede derivar en una cadena que comienza con ese mismo símbolo A (es decir, $A \rightarrow^+ Aa$). Esta patología es fatal para los analizadores sintácticos descendentes (como los de descenso recursivo), ya que provoca que el analizador entre en un bucle infinito al intentar expandir el mismo no terminal repetidamente sin consumir ningún símbolo de la entrada.

Ejemplo Práctico:

Supongamos la gramática para listas de términos: $A \rightarrow A\alpha \mid \beta$, donde α y β son secuencias de símbolos y β no comienza con A . Algoritmo paso a paso para eliminarla:

- 1) Agrupación: Se identifican las producciones del no terminal A y se separan en dos grupos: las que presentan recursividad inmediata ($A\alpha_1, A\alpha_2, \dots$) y las que no ($\beta_1, \beta_2, \dots$).
- 2) Reescritura del no terminal original: Se reemplazan las producciones de A de manera que ahora comiencen con los casos base (β) seguidos de un nuevo símbolo no terminal A' .

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

- 3) Definición del nuevo no terminal: Se define A' para que consuma los sufijos recursivos (α) y se añade la producción vacía ϵ para finalizar la recursión.

$$A \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$

Resultado Optimizado:

Aplicado a la regla clásica de expresiones $E \rightarrow E + T \mid T$, la gramática sin recursividad por la izquierda resultante es: $E \rightarrow TE'$ y termina en: $E' \rightarrow +TE' \mid \epsilon$

c) Factorización por la Izquierda

La factorización por la izquierda es una transformación gramatical necesaria cuando dos o más producciones de un mismo no terminal comparten un prefijo común. Al analizar la entrada de izquierda a derecha, el analizador no tiene forma de saber cuál de las alternativas elegir hasta que haya procesado todo el prefijo común.

Ejemplo Práctico:

Consideremos la abstracción del clásico problema del else colgante:

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

En este caso, i es "if", E es la condición, t es "then", S es la sentencia y e es "else". Ambas opciones condicionales comparten el prefijo $iEtS$.

Gramática Optimizada:

El algoritmo dicta encontrar el prefijo común más largo α en $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ y diferir la decisión factorizando α mediante un nuevo no terminal A' de la forma $A \rightarrow \alpha A'$ y $A' \rightarrow \beta_1 \mid \beta_2$. Aplicando esto, la gramática factorizada por la izquierda es:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

4.- De la Expresión al Autómata (Caso Práctico: Ajedrez).

4.1.- Notación PGN (Portable Game Notation).

La notación PGN (Portable Game Notation) es un estándar basado en texto para registrar partidas de ajedrez. Su sintaxis completa es compleja y contextualmente sensible, pero los movimientos individuales (tokens) siguen patrones estrictamente regulares, lo que los

hace perfectos para ser analizados mediante expresiones regulares y Autómatas Finitos Deterministas (Aho et al., 2007).

Para este caso práctico, definiremos un subconjunto simplificado del lenguaje PGN que reconozca exclusivamente movimientos básicos de piezas y peones, con capturas y jaques opcionales.

Reglas del subconjunto:

- 1) Pieza (Opcional): Representada por una letra mayúscula inglesa correspondiente a Rey, Dama, Torre, Alfil o Caballo. Si se omite, se asume que es un peón.
- 2) Captura (Opcional): Representada por el carácter de la letra equis minúscula.
- 3) Columna (Obligatorio): Letra minúscula inglesa de la primera a la octava letra del alfabeto.
- 4) Fila (Obligatorio): Dígito del uno al ocho.
- 5) Jaques / Mates (Opcional): Representado por el signo de suma o el símbolo de numeral.

Alfabeto Formal (Σ):

$$\Sigma = \{K, Q, R, B, N, x, a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8, +, \#\}$$

4.2.- Subconjunto simplificado del PGN para movimientos básicos.

En la teoría de la computación, las expresiones regulares son una notación algebraica que define lenguajes regulares mediante operaciones de unión, concatenación y cerradura de Kleene (Sipser, 2013).

Aplicando las reglas de nuestro subconjunto, construimos las sub-expresiones:

- Piezas = $(K \cup Q \cup R \cup B \cup N \cup \epsilon)$
- Capturas = $(x \cup \epsilon)$
- Columna = $(a \cup b \cup c \cup d \cup e \cup f \cup g \cup h)$
- Fila = $(1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6 \cup 7 \cup 8)$
- EstadoFinal = $(+ \cup \# \cup \epsilon)$

La concatenación secuencial de estas sub-expresiones genera la Expresión Regular (R) para nuestro lenguaje:

$$R = \text{Pieza} \cdot \text{Captura} \cdot \text{Columna} \cdot \text{Fila} \cdot \text{EstadoFinal}$$

El símbolo ϵ representa la cadena vacía, denotando que el elemento es opcional (Linz, 2016).

4.3.- Expresión Regular (Regex) y el Autómata Finito Determinístico (AFD) equivalente.

Un teorema fundamental de la teoría de lenguajes establece que toda expresión regular tiene un autómata finito equivalente capaz de reconocer el lenguaje que describe (Hopcroft y Ullman, 2006).

- Definimos matemáticamente nuestro AFD como la 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$:
- Estados (Q): $\{q_0, q_1, q_2, q_3, q_4, q_5, q_{err},\}$

- Alfabeto (Σ): definido en la sección 1
- Estado inicial (q_0): q_0
- Estados de aceptación (F): $\{q_4, q_5\}$ (El movimiento es válido al tener una coordenada completa, con o sin jaque).

Función de Transición (δ):

La función $\delta : Q \times \Sigma \rightarrow Q$, dicta el cambio de estados. Cualquier transición no listada conduce al estado pozo/muerto q_{err} para rechazar cadenas inválidas

- $\delta(q_0, [KQRBN]) = q_1$ (lee pieza mayor)
- $\delta(q_0, [a - h]) = q_3$ (avanza directamente a la columna si es peon)
- $\delta(q_1, x) = q_2$ (lee captura tras una pieza)
- $\delta(q_1, [a - h]) = q_3$ (lee columna tras una pieza sin captura)
- $\delta(q_2, [a - h]) = q_3$ (lee columna tras captura)
- $\delta(q_3, [1-8]) = q_4$ (lee fila, estado de aceptación)
- $\delta(q_4, [+ \#]) = q_5$ (lee jaque/mate, estado de aceptación)

Conclusión

A lo largo de este informe, se ha demostrado de manera teórica y práctica que la capacidad de una máquina para interpretar y procesar información depende directamente de la precisión con la que se definen sus reglas gramaticales y sus fronteras computacionales.

En primer lugar, la Jerarquía de Chomsky provee un mapa esencial para el diseño de sistemas informáticos, evidenciando que a mayor restricción gramatical, mayor es la eficiencia del algoritmo de reconocimiento. Esto quedó materializado al modelar trayectorias espaciales mediante Gramáticas Libres de Contexto y al reconocer tokens de ajedrez (PGN) empleando expresiones regulares y Autómatas Finitos Deterministas. Se comprobó que, si bien las GLC permiten el anidamiento y la bifurcación (necesarias para fractales o estructuras de control), los autómatas finitos ofrecen la velocidad lineal requerida para el análisis léxico estricto.

En síntesis, la correcta sinergia entre gramáticas bien estructuradas y autómatas eficientes es lo que hace posible la existencia de lenguajes de programación, motores de búsqueda, protocolos de red y compiladores. Dominar estos fundamentos es dominar el lenguaje interno mediante el cual la humanidad instruye y controla a los sistemas computacionales.

Referencias Bibliografía

- Aho, A. V. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools*. Pearson/Addison-Wesley.
- Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124.
- Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control*, 2(2), 137-167.
- Harrison, M. A. (1978). *Introduction to Formal Language Theory*. Addison-Wesley.
- Hopcroft, J. E. y Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computations*. Prentice-Hall.
- Linz, P. (2016). *An Introduction to Formal Languages and Automata*. Jones & Bartlett Learning.
- Martin, J. C. (2011). *Introduction to Languages and the Theory of Computation*. McGraw-Hill.
- Sipser, M. (2013). *Introduction to the Theory of Computation*. Cengage Learning.
- Rozenberg, G. y Salomaa, A. (Eds.). (1997). *Handbook of Formal Languages*. Springer.